

EduReach: A Low-Bandwidth Virtual Classroom with Blockchain-Verified Credentials for Rural Technical Education

Sep 28, 2026 · @Hanzala

EduReach delivers expert instruction to rural diploma colleges over connections that conventional video conferencing cannot use, at roughly 10.6 MB per student per lecture-hour instead of 439 MB, and issues completion certificates whose authenticity an employer can verify without trusting the issuing college's server. A working implementation exists: 7,800 lines across five services, 48 automated tests, and an estimated running cost of Rs 2,500 to 6,000 per month for an institute of 200 students.

The problem

The barrier is infrastructure, not willingness. Rural diploma colleges routinely operate without subject lecturers in fields such as artificial intelligence, VLSI and renewable energy. Expert faculty exist in the cities and are often willing to teach remotely. What fails is the link between them.

Conventional video conferencing assumes bandwidth that a village campus does not have. The arithmetic is unforgiving: a one-hour class at 480p costs about 439 MB per student, so a cohort of 200 pulls roughly 87.8 GB in a single session. Even at 144p it is 26.4 GB. On an unstable rural connection, and on student data plans, that is not a quality problem but a hard impossibility.

The consequences compound. Students fall back on self-study material or travel to a city for coaching, which costs money and days. The institute cannot offer the electives its affiliation permits, so students graduate without the subjects employers now ask for. The urban-rural gap widens each year.

Earlier interventions failed for three recurring reasons, and each is a design constraint here:

- They assumed a high-bandwidth link and degraded into unusable stalling when it was absent.
- They required specialised equipment or studio setups the institute could not fund or maintain.
- They imposed a learning curve that pushed faculty back to the telephone and the photocopier.

There is a fourth problem that sits behind the first three and is rarely addressed. Even when a rural student completes a course, the certificate is worth what a verifier believes it is worth. Checking one means emailing a registrar and hoping for a reply. That asymmetry is a real cost borne by exactly the students this kind of programme is meant to help.

Who it serves

Three groups, with different needs and one shared constraint.

Group	What they need	What the system gives them
Rural diploma students	Instruction in subjects their college cannot staff, on an entry-level phone and a limited data plan	Audio-first lectures at about 10.6 MB per hour, downloadable overnight, playable with no signal
Urban subject faculty	To teach remotely from any quiet room, without a studio or a training course	One-click live sessions that open in audio mode; upload a recording and the system does the rest
Institutes and boards	To widen their course catalogue without capital spending, and to issue certificates that hold up outside the college	Software-only deployment on a small cloud instance; ledger-anchored credentials verifiable by anyone

The design target is the hardest case rather than the average one: a student on an entry-level Android phone, on a shared or prepaid data plan, on a connection that drops several times an hour. A system that works there works everywhere else. The reverse is not true, which is why systems designed for the average case keep failing in the villages.

A fourth group matters for the credential layer. Employers, other colleges and scholarship bodies need to check a certificate without an account on this platform and without trusting the issuing institute's server. They are served by a public verification endpoint that answers from the ledger.

What EduReach is

EduReach is a software-only virtual classroom in which the network is assumed to be absent and its presence treated as an optimisation. It runs as a progressive web app on any phone with a browser, backed by a small cloud server, and needs no specialised hardware and no per-seat licences.

Four commitments shape every component:

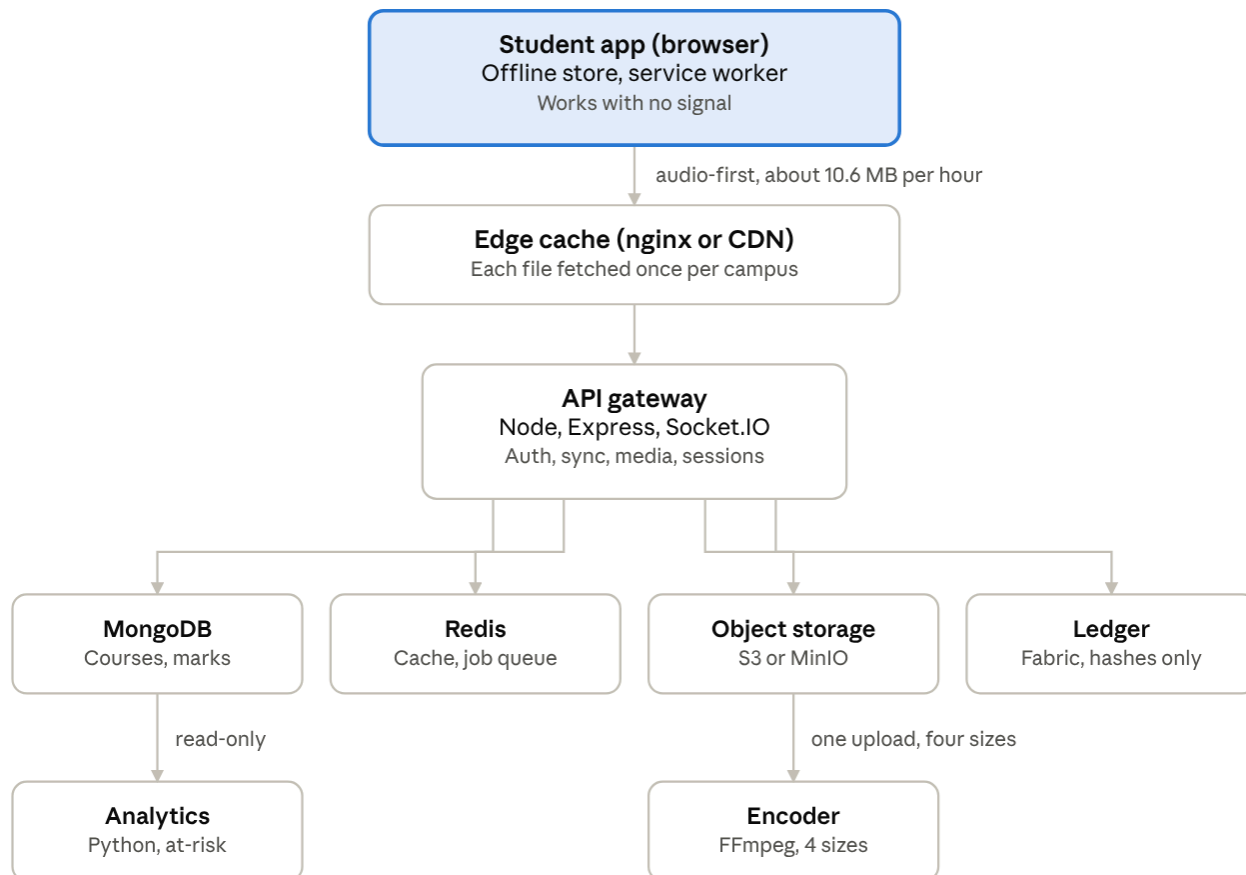
1. **Audio first, always.** A lecture is delivered as compressed speech by default. Video is an option the student opts into after being told what it will cost them in megabytes.
2. **The device is the source of truth for the student.** Lectures, quizzes and discussion threads live on the phone. Screens read from local storage and refresh from the network when there is one, so nothing ever waits on a request that may never return.
3. **Writes are queued, never blocking.** Every student action is recorded locally with an idempotency key, shown immediately, and delivered when a connection appears. A quiz answered at 9 pm offline is recorded as 9 pm, not as the moment it finally syncs.
4. **The control plane is separate from media.** A live class sends slide indices and poll deltas over a socket, measured in tens of bytes. Media is a separate stream at a bitrate the student chose. A student on 40 kbps stays in the room and follows the slides.

On top of this sit two capabilities that distinguish it from a compressed video platform. Academic credentials are anchored on a distributed ledger, so a certificate can be checked by a third party without trusting this server. And a learning-analytics service identifies students falling behind academically while explicitly refusing to treat poor connectivity as a risk signal.

System architecture

Only MongoDB is required; the rest degrade rather than fail

Encoding runs off the request path, so a slow upload never blocks a live class



system architecture · 9 components

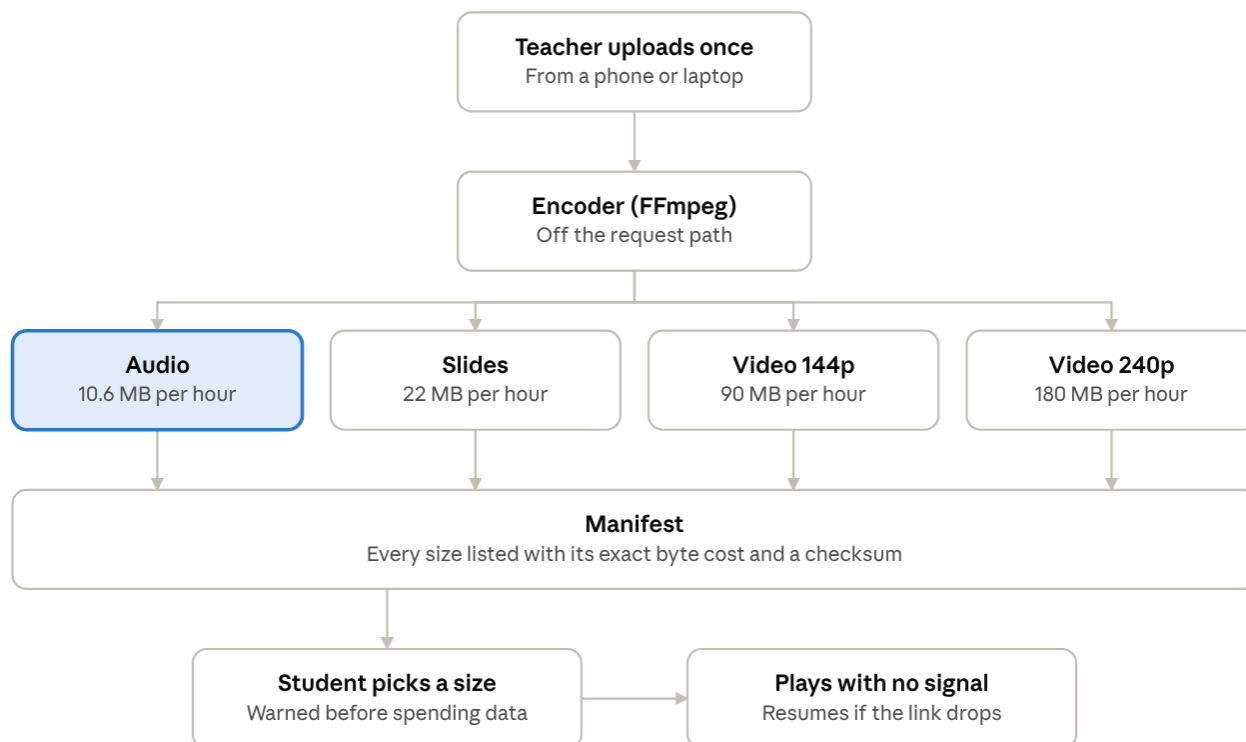
MongoDB is the only hard dependency. If Redis is unreachable the API keeps serving and caching falls through to the database; if object storage is down, media requests return a retry signal rather than an error, and the rest of the system is unaffected. This matters because a rural deployment cannot assume an operator is watching.

The encoder runs in its own process. A twenty-minute FFmpeg job must never make the API unresponsive, and capacity is added by running more encoder containers rather than a bigger server. The analytics service reads the database directly and never writes to it, so a fault there cannot corrupt academic records.

How a lecture reaches a student

One upload becomes four sizes; the student chooses what to spend

Audio is built and published first, so a lecture is usable before the video finishes encoding



delivery pipeline · upload to playback

The teacher does nothing beyond uploading. There are no encoding settings, no resolution choices and no file preparation, because every one of those is a place where a busy lecturer abandons the tool. The compression decisions are made once, in code, and applied to every upload.

Each size is published the moment it finishes rather than at the end of the batch. Audio is built first, so students can begin downloading a usable lecture while the 240p version is still encoding. If a later size fails, students still have the earlier ones instead of nothing.

The encodings themselves are deliberately conservative. Audio is Opus at 24 kbps in mono with silence trimmed, which is enough for speech and nothing more. Slides mode detects scene changes and stores each as a compressed still image, so a diagram costs a few kilobytes once instead of fifteen frames a second forever. Video uses the H.264 baseline profile at 15 to 20 frames per second, because lecture footage has little motion and a baseline profile decodes on the cheapest hardware.

Working without a connection

A student who has no signal for an evening loses nothing. Take the case the system is designed around: a student answers a quiz at 9 pm with the network down, posts a question on the discussion board, and finds a signal the next morning.

1. The quiz is graded against a copy already on the phone and the answers are queued locally with a unique operation identifier. The screen says plainly that the work is saved and will be sent later, rather than showing a failure.
2. The discussion post appears in the thread immediately, marked *waiting to send*.
3. In the morning the app measures the connection. It does not trust the operating system's online flag, because a phone attached to a tower with dead backhaul reports itself as online.
4. The queue is delivered in one batch. The server records both actions against their original timestamps, so the quiz is recorded as 9 pm the previous evening.
5. If the response is lost in transit and the phone sends again, the server recognises the operation identifier and returns the original result instead of creating a second attempt.

The fifth point is what makes the design safe rather than merely convenient. Over a flaky link, duplicate delivery is not an edge case but the normal condition, so every operation is written to be applied repeatedly without changing the outcome. This was verified by sending the same batch of three operations three times: the first produced two applied results, the second and third reported them as duplicates, and exactly one attempt existed in the database afterwards.

One failure inside a batch does not discard the rest of it. A reply to a deleted thread should not cost a student their quiz result, so each operation is judged on its own and the client is told which ones are worth retrying. An operation that can never succeed is reported as such, so the device stops spending data on it.

Bandwidth and efficiency

The default mode costs a student about 10.6 MB per lecture-hour. A conventional platform streaming the same hour at 480p and 1 Mbps costs 439 MB, so the default is roughly forty times cheaper. The encoding ladder below is the whole of it; data per student is bitrate multiplied by duration, divided by eight.

Mode	Encoding	Per student, 1 hour	200 students, 1 hour	Where it is used
Audio only	Opus 24 kbps, mono, silence trimmed	10.6 MB	2.1 GB	The default everywhere
Slides and audio	Scene-detected WebP stills plus audio	22 MB	4.3 GB	Diagram-heavy lectures
Video 144p	H.264 baseline, 200 kbps, 15 fps	90 MB	18 GB	Scheduled overnight download
Video 240p	H.264 baseline, 400 kbps, 20 fps	180 MB	36 GB	Campus wifi or a good 4G cell
Video 480p	H.264 main, 900 kbps, 25 fps	400 MB	78 GB	Opt-in only, with a warning

Slides mode deserves attention because it is where the largest saving hides. A lecturer talking over diagrams is visually static for minutes at a time. Storing each diagram once as a compressed still, and sending it only when it changes, costs a few kilobytes per slide instead of fifteen frames a second indefinitely. It delivers the pedagogical content of video at roughly a quarter of 144p's cost.

Seven further measures compound the saving, and each is implemented rather than planned:

- **Measured mode selection.** The client times a round trip to a small endpoint and the server recommends the richest mode the link can carry. The operating system's connectivity flag is not trusted.
- **A control plane separate from media.** A live class sends slide indices, poll deltas and chat lines, in tens of bytes. A poll update carries the changed count, not the question.
- **Edge caching of immutable media.** Derived files never change, so they are cached for a year. One campus fetches a lecture from the origin once, not once per student.
- **Resumable ranged downloads.** Progress is written to the device every 512 KB. A connection lost at 80 percent resumes at 80 percent, which on a 2G link is the difference between a lecture that eventually arrives and one that never does.
- **Checksum verification on the device.** A file corrupted in transit is detected before playback, so it is re-fetched deliberately rather than discovered broken mid-lecture.
- **Cursor-based synchronisation.** A returning device asks for everything after the last item it holds. The payload is proportional to how long it was offline, not to how much exists.

- **A small client.** The whole application is 85 KB compressed, and a three-question quiz is delivered in 746 bytes.

The cumulative effect is that a 200-student cohort can take a full hour of live instruction for about 2.1 GB of downstream data across the entire campus, which fits comfortably inside an ordinary broadband connection and inside student data plans.

Verifiable credentials

Anchoring a certificate on a ledger changes the question a verifier must answer. Instead of *do you trust this college's server?* it becomes *does this fingerprint appear on the ledger?*, which anyone can check, including after the server is switched off.

No personal data reaches the ledger. Not names, not phone numbers, not marks. A ledger is append-only and replicated to every participating organisation, which makes it precisely the wrong place for student data: it cannot be corrected, cannot be deleted, and is visible to every participant. What is stored per credential is an identifier, a SHA-256 fingerprint, the issuing institute and the date. Student and course references are keyed hashes of internal identifiers, so the ledger cannot be mined for a student roster even by a participating college. The real record, with the name and the grade, stays in the institute's own database.

Verification runs in four steps, and none of them requires trusting this platform:

1. The holder presents the certificate file, which the app saves for them.
2. Its contents are reduced to a canonical form, with keys sorted at every depth, so that any verifier reproduces the same byte string.
3. The SHA-256 of those bytes must equal the recorded fingerprint.
4. That fingerprint must resolve on the ledger and must not be revoked.

Changing a single character of the name or the grade changes the fingerprint, and step three or four fails. This was tested directly: editing an average from 78 to 95 and resubmitting produced a different digest and the response *hash is not present on the ledger*. Revocation appends a new entry rather than rewriting the old one, so the history shows both the issue and the revocation with their own timestamps.

Two ledger backends sit behind one interface, and the choice is a configuration flag rather than a rewrite.

	Local hash chain (default)	Hyperledger Fabric
Infrastructure	None beyond the existing database	Peers, ordering service, certificate authority

	Local hash chain (default)	Hyperledger Fabric
Guarantee	Tamper-evident: any edit breaks every subsequent hash	Tamper-resistant: rewriting requires collusion across organisations
Suited to	A single institute, a pilot, day-one deployment	A university consortium or a board
Cost	Included in the base server	Additional nodes and operational staff

The hash chain exists because of the brief's own constraint. Requiring a rural college to stand up and *operate* a Fabric network in order to issue certificates would make the feature dead on arrival. It runs on the same small server as everything else, and every block commits to its predecessor, so editing any historical row invalidates every hash after it. The honest limitation is stated in the limitations section: it is tamper-evident, not tamper-proof.

When Fabric is enabled and unreachable, the service refuses to start rather than silently falling back. An institute that asked for consortium-backed credentials must not be handed weaker ones without being told.

Learning analytics

The model answers one question for a teacher who cannot see the room: which students are quietly falling behind and need contacting before they drop out. It runs as a separate Python service that reads the database and never writes to it.

It is unsupervised by necessity. A new deployment has no labelled dropouts to train on, and by the time it does, the students it would have helped are already gone. The service clusters each cohort on academic signals and flags the cluster whose centre sits worst. Below six students it falls back to transparent rules, because clustering four people produces groups that mean nothing.

Connectivity is deliberately excluded from the feature set. Preferred playback mode, proportion of work done offline and data consumption are all available and all withheld from the model. A student on audio-only is adapting correctly, not falling behind. Including those signals would train the system to flag poverty rather than academic difficulty, in a programme whose entire purpose is to serve students on bad connections.

This is enforced by a test rather than by intention. Eight students, academically identical, four working entirely offline on audio and four on video with good signal: the test asserts the offline group scores no higher on risk than the others, and a second test asserts that no connectivity field appears in the feature list at all.

Every flag carries a plain-language reason, so a teacher can act without trusting a number they cannot interrogate. Working offline appears among those notes, but framed as context rather than concern: *works mostly offline, so contact by phone may reach them faster*.

Technology stack

Every component is open-source with a permissive licence. There are no per-seat fees, no proprietary codecs and no hardware dependencies.

Layer	Technology	Why this one
Client	React, built with Vite, as a progressive web app	Installs from a browser with no app store, and works on any Android phone. 85 KB compressed
Offline storage	IndexedDB and a service worker	Lets the application open and play saved lectures with no network at all
API	Node.js with Express	One language across client and server, and a large pool of graduates who can maintain it
Realtime	Socket.IO	Falls back to long polling where a carrier proxy breaks WebSocket upgrades, which is common on rural networks
Database	MongoDB	Flexible records for evolving curricula, and a single node is sufficient at institute scale
Cache and queue	Redis	Serves the encoding queue and read cache; the system degrades rather than fails without it
Object storage	MinIO, or S3-compatible cloud	MinIO on the same server for a pilot, cloud storage at scale, with no code change
Media encoding	FFmpeg with Opus and H.264	The reference open-source encoder; Opus is unmatched for speech below 32 kbps

Layer	Technology	Why this one
Ledger	Hyperledger Fabric, or a built-in hash chain	Fabric for a consortium; the hash chain so the feature works on day one without one
Analytics	Python with pandas and scikit-learn	The standard stack for this work, and familiar to any statistics graduate
Edge	nginx	Caching and compression at the campus or CDN edge

The stack is chosen as much for who can maintain it as for what it does. A rural institute cannot recruit specialists, so every component is one that an ordinary computer science or IT diploma holder has already met.

Build status

This is a working implementation, not a design study: roughly 7,800 lines across five services, with 48 automated tests. The status is set out honestly below, because a funder is better served by knowing what remains than by a uniform claim of completion.

Capability	Status	Evidence
Authentication, courses, enrolment	Working	Driven end to end in a browser
Offline storage and playback	Working	Application opens and reads with the network disabled
Offline queue and synchronisation	Working and tested	Post written offline delivered on reconnect; triple send produced one record
Quizzes, polls, discussion	Working	Graded server-side; answer keys confirmed absent from the student payload
Live sessions and realtime	Working	Slide, poll and chat messages verified over the socket
Credential issue, revoke, verify	Working and tested	Forged payload rejected; chain integrity held across all writes
Learning analytics	Working	Ran against live data through the gateway

Capability	Status	Evidence
Encoding pipeline	Implemented, not yet proven	Requires FFmpeg and object storage; not exercised against a real video file
Hyperledger Fabric backend	Implemented, not yet deployed	Chaincode written; not run against a live Fabric network

The two unproven items are the honest gaps. The encoding pipeline and the Fabric deployment both need infrastructure that was unavailable on the development machine, and neither should be treated as validated until a pilot exercises them. They are the first milestone in the roadmap for exactly that reason.

The automated tests concentrate on the properties that would be expensive to get wrong: that a ledger detects tampering, including an attacker who recomputes a block hash to hide an edit; that twenty-five concurrent credential issuances produce no duplicate block; that a resent operation batch cannot duplicate a student's work; that an offline timestamp survives synchronisation; and that the at-risk model does not penalise students on poor connections.

Cost

Running EduReach for one institute of 200 students costs an estimated Rs 2,500 to 6,000 a month in infrastructure, or Rs 7,500 to 16,000 including maintenance and support. These are planning estimates at 2026 cloud list prices and should be confirmed at procurement; the basis for each line is given so the arithmetic can be checked.

Recurring, one institute of 200 students

Item	Monthly (INR)	Basis
Compute	1,000 – 2,000	One 2 GB instance running the API and the encoder
Storage	500 – 1,000	10 to 50 GB of compressed renditions and backups
Data transfer	1,000 – 3,000	About 42 GB a month at audio-first defaults, after edge caching
Infrastructure subtotal	2,500 – 6,000	

Item	Monthly (INR)	Basis
Maintenance and support	5,000 – 10,000	Patching, monitoring, helpdesk
Total	7,500 – 16,000	

The transfer figure is where the engineering pays. Two hundred students taking twenty lecture-hours a month consume about 42 GB in audio-first mode. The same usage at 480p would be roughly 1.6 TB, close to forty times the egress and the bill that goes with it.

One-time

Item	Cost (INR)	Notes
Platform development	50,000 – 2,00,000	Already completed; a working implementation accompanies this proposal
Pilot deployment and field testing	1,00,000 – 2,00,000	Two institutes, one semester, including the two unproven components
Faculty orientation	25,000 – 50,000	Half a day per institute; the interface is deliberately minimal
Security and accessibility audit	50,000 – 1,00,000	Before any wider rollout

Cost per student falls sharply with scale

Deployment	Students	Monthly infrastructure (INR)	Per student per year (INR)
Single institute	200	2,500 – 6,000	150 – 360
District cluster, 10 institutes	2,000	12,000 – 25,000	72 – 150
State platform, 100 institutes	20,000	80,000 – 1,60,000	48 – 96

The reason for the fall is that content is shared. A lecture on VLSI design recorded once by one expert serves every institute on the platform, so the storage and encoding cost is paid once for the whole state rather than once per college. Adding an institute adds data

transfer and almost nothing else. This is the opposite of the per-seat licensing model that makes commercial platforms unaffordable at scale, and it is why a state-level deployment costs less per student than a single college running alone.

Expected impact

The measurable effect is that a subject a college cannot staff becomes a subject it can offer, at a data cost its students can afford.

For students. A full hour of expert instruction costs about 10.6 MB instead of 439 MB, so a month of twenty lecture-hours costs roughly 212 MB rather than 8.8 GB. Learning continues through connectivity gaps rather than stopping at them, and travel to a city for coaching becomes unnecessary for the subjects covered. Completion is recorded as a certificate the student can carry and a third party can check.

For faculty. An expert can teach a remote cohort from any quiet room with a phone, without a studio, a training course or an operator. One-click session start and automatic compression remove the two steps where remote-teaching tools are usually abandoned. The at-risk view gives a teacher who cannot see the room the one thing they lose by teaching remotely: knowing who is struggling.

For institutes. The catalogue widens without capital spending. A college can offer artificial intelligence or VLSI electives by drawing on the platform's shared library rather than by recruiting a specialist it cannot fund or retain. Certificates gain standing outside the institution without the registrar answering verification requests by email.

For the system as a whole. Expertise is decoupled from location. One lecturer recorded once reaches every participating college, which changes the economics of specialist teaching from one-expert-per-college to one-expert-per-state. The credential ledger creates an auditable record of what was actually taught and completed, which is useful to a funding body evaluating outcomes.

Two cautions belong here rather than in a later section. First, these are engineering outcomes; learning outcomes depend on teaching quality and student motivation, neither of which software supplies. Second, the figures above are the system's measured or derived costs, not results from a field deployment. Establishing effect on completion and pass rates is the purpose of the pilot.

Policy alignment

The proposal addresses stated national priorities in digital education access rather than creating a new category of spending.

- **Equitable access to quality technical education.** The National Education Policy 2020 places repeated emphasis on bridging the digital divide and on technology that works within existing infrastructure constraints. This system is built for the constraint rather than around it.
- **Digital public infrastructure.** The deployment is open-source, vendor-neutral and free of per-seat licensing, so a state can own and operate it rather than renting it. There is no lock-in to a commercial platform and no recurring licence liability.
- **Verifiable academic records.** Ledger-anchored credentials align with the direction of national digital credentialing, and the design keeps personal data off the ledger entirely, which is the correct posture under data protection obligations.
- **Skilling and employability.** The subjects the system is designed to deliver, such as artificial intelligence, VLSI and renewable energy, are precisely those where rural diploma colleges cannot recruit and where industry demand is growing.
- **Effective use of existing connectivity.** The system is designed to work on the connectivity rural institutions already have, rather than requiring an upgrade first. Where public broadband has reached a campus, the system uses it well; where it has not, the system still functions.

A funded pilot would produce something a policy body currently lacks: measured data on what remote technical instruction actually costs per student, in megabytes and in rupees, on real rural links.

Limitations and risks

Stated plainly, because a reviewer will find these anyway and the proposal is stronger for naming them first.

The default ledger is tamper-evident, not tamper-proof. The built-in hash chain makes any alteration detectable, but an administrator with database access could in principle rewrite the chain from its origin. Only the Hyperledger Fabric backend, with peers at more than one organisation, makes rewriting require collusion. Deployments that need that guarantee should budget for it; the design supports the switch without code changes.

A ledger proves issuance, not merit. It can show that a certificate was issued by a named institute and has not been altered. It cannot show that the student earned it. Anchoring a poorly assessed course preserves a poor assessment immutably.

Two components are implemented but unproven. The encoding pipeline has not been run against a real video file, and the Fabric chaincode has not been deployed to a live network. Both need infrastructure a pilot would provide, and both carry the ordinary risk that first contact with reality reveals defects. The bandwidth figures are derived from the configured encoding parameters, not measured from encoded output.

Audio-first teaching does not suit every subject. A lecture that depends on watching a hand move, a laboratory demonstration or a physical process, loses something real when compressed to speech and stills. The system should be positioned for theory-heavy subjects, where the loss is small, rather than as a universal replacement for the classroom.

Device availability is assumed. The design targets an entry-level smartphone, but it assumes each student has one. Where they do not, a shared campus device model works but changes the offline and privacy behaviour, and sign-out wipes the device precisely for that reason.

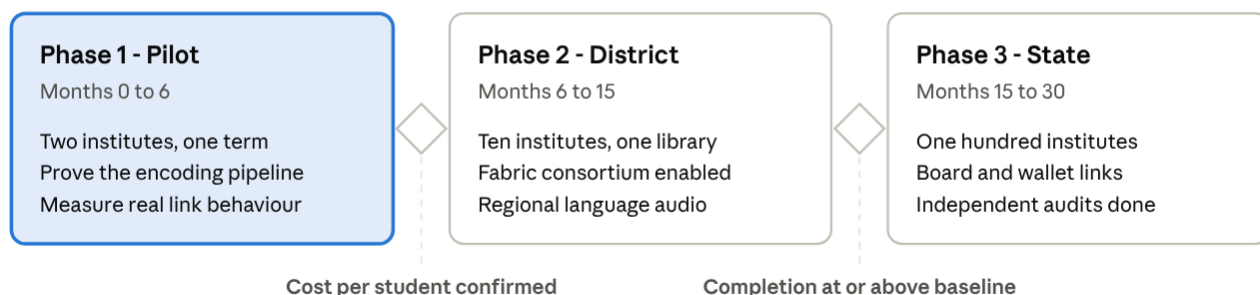
The analytics model is unvalidated. It is unsupervised and rule-based by necessity, and its fairness property is enforced by tests, but its actual predictive value is unknown until a cohort has been followed through a full term.

Adoption is the largest risk, and it is not technical. Every prior intervention cited in the problem statement failed at least partly on human grounds. Faculty time, institutional incentive and student trust will determine the outcome more than any engineering decision in this document.

Roadmap and further work

Each phase is gated on evidence from the one before

Equal bands, not drawn to scale



roadmap · 3 phases, 2 gates

The funding ask is Phase 1. It closes the two open gaps in the build status table by exercising the encoding pipeline and the Fabric deployment against real infrastructure, and it produces the measurement a policy body currently lacks: what remote technical instruction actually costs per student on real rural links.

Beyond the three phases, several extensions follow naturally from the architecture and would each be cheap to add:

- **Regional language delivery.** Speech is already the primary channel, and a transcript costs a few kilobytes. Machine-assisted transcripts and translated captions would let one recorded lecture serve several language regions at almost no additional bandwidth.
- **Campus-local caching.** A small always-on device on the campus network could hold the current week's lectures, so students download over local wifi and the institute's uplink carries each file exactly once. This is the single largest remaining saving available.
- **Phone-to-phone sharing.** Students on the same campus could pass verified lecture files between devices over local wireless, with the existing checksum guaranteeing integrity. A student with no data at all could then still receive a lecture.
- **A voice and SMS fallback.** For students without a smartphone, the audio track is already the product; delivering it by a call-in line, with quizzes over SMS, would extend reach to the hardest-to-serve group.
- **Credential wallet integration.** The exported certificate is a self-contained, independently verifiable file, which makes integration with a national digital document wallet a matter of format rather than architecture.
- **A shared expert pool.** The content library is already state-shareable. A directory letting any participating college book a remote specialist for a live session would turn the platform from a content store into a teaching exchange.
- **Accessibility work.** Screen-reader passes, captioning and a low-literacy voice navigation mode are not yet done and should be funded as part of the audit in Phase 3.

The common thread is that each of these rides on decisions already made: audio as the primary channel, files that verify themselves, and content that is shared rather than duplicated.